



Facultad de Ingeniería

Diseño de Aplicaciones 2

Obligatorio 2 – Descripción del diseño

“Entregado como requisito para la obtención del título de Ingeniero en Sistemas”

Repo: <https://github.com/IngSoft-DA2/271395-294598-221509.git>

Candelaria Blanco - 271395

Maximiliano Gimenez - 294598

Ana Betina Kadessian - 221509

Tutores:

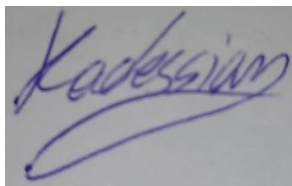
Nicolás Fierro, Maximo Healty

2025

DECLARACIÓN DE AUTORÍA

Nosotros, Candelaria Blanco, Maximiliano Gimenez y Ana Betina Kadessian, declaramos que el trabajo que se presenta en esa obra es de nuestra propia mano. Podemos asegurar que:

- La obra fue producida en su totalidad mientras realizábamos el obligatorio 2 de la materia Diseño de Aplicaciones 2.
- Cuando hemos consultado el trabajo publicado por otros, lo hemos atribuido con claridad
- Cuando hemos citado obras de otros, hemos indicado las fuentes. Con excepción de estas citas, la obra es enteramente nuestra
- En la obra, hemos acusado recibo de las ayudas recibidas
- Cuando la obra se basa en trabajo realizado conjuntamente con otros, hemos explicado claramente qué fue contribuido por otros, y qué fue contribuido por nosotros
- Ninguna parte de este trabajo ha sido publicada previamente a su entrega, excepto donde se han realizado las aclaraciones correspondientes.



Ana Betina Kadessian

18/6/2025



Candelaria Blanco

18/6/2025



Maximiliano Gimenez

18/6/2025

ABSTRACT

Este trabajo presenta el desarrollo de un Simulador de Objetos que permite modelar y ejecutar estructuras propias de la Programación Orientada a Objetos (POO). El sistema posibilita la creación de clases con herencia simple, definición de atributos y métodos, y ejecución de métodos respetando accesibilidad, herencia y polimorfismo. La solución fue implementada como una API REST utilizando .NET 8 y Entity Framework Core, siguiendo buenas prácticas de diseño, Clean Code y TDD y un frontend en Angular. Este proyecto fue desarrollado para la Universidad ORT Uruguay como parte de la materia Diseño de Aplicaciones 2, con el objetivo de aplicar conceptos avanzados de diseño de software en un entorno práctico y educativo.

Índice

Descripción general.....	5
Qué hace la solución.....	5
Vista de desarrollo o componentes modelo 4+1	6
Diagrama general de paquetes	6
Breve descripción de responsabilidades de cada paquete.....	7
Diagrama de componentes	8
Vista Lógica o Diseño Modelo 4+1	9
Diagrama de clases de cada paquete	9
Domain:.....	9
IAdapter:.....	10
Adapter:.....	10
IBusinessLogic:	11
BusinessLogic:	11
IDataAcces:	11
DataAcces:	12
WebApi.....	12
Models:.....	13
ServiceFactory:	13
TransformerAbstractions:.....	13
Funcionalidades de mayor relevancia	14
Diagramas de interacción que muestran las clases involucradas	14
Descripción de jerarquías de herencia utilizadas.....	15
Modelo de tablas de la estructura de la base de datos	16
Justificación del diseño	17
Mecanismos de inyección de dependencias	17
Principios SOLID	17
Otros principios de diseño	18
Decisiones propias de diseño	19
Métricas principios a nivel de paquetes	20
Mejoras en las métricas de paquetes.....	23
Descripción del manejo de excepciones	23

Implementación de un Nuevo Transformador Reflection	23
Anexo	25
Especificación API	25
Informe de cobertura.....	30
Configuración de la aplicación de angular:.....	34

Descripción general

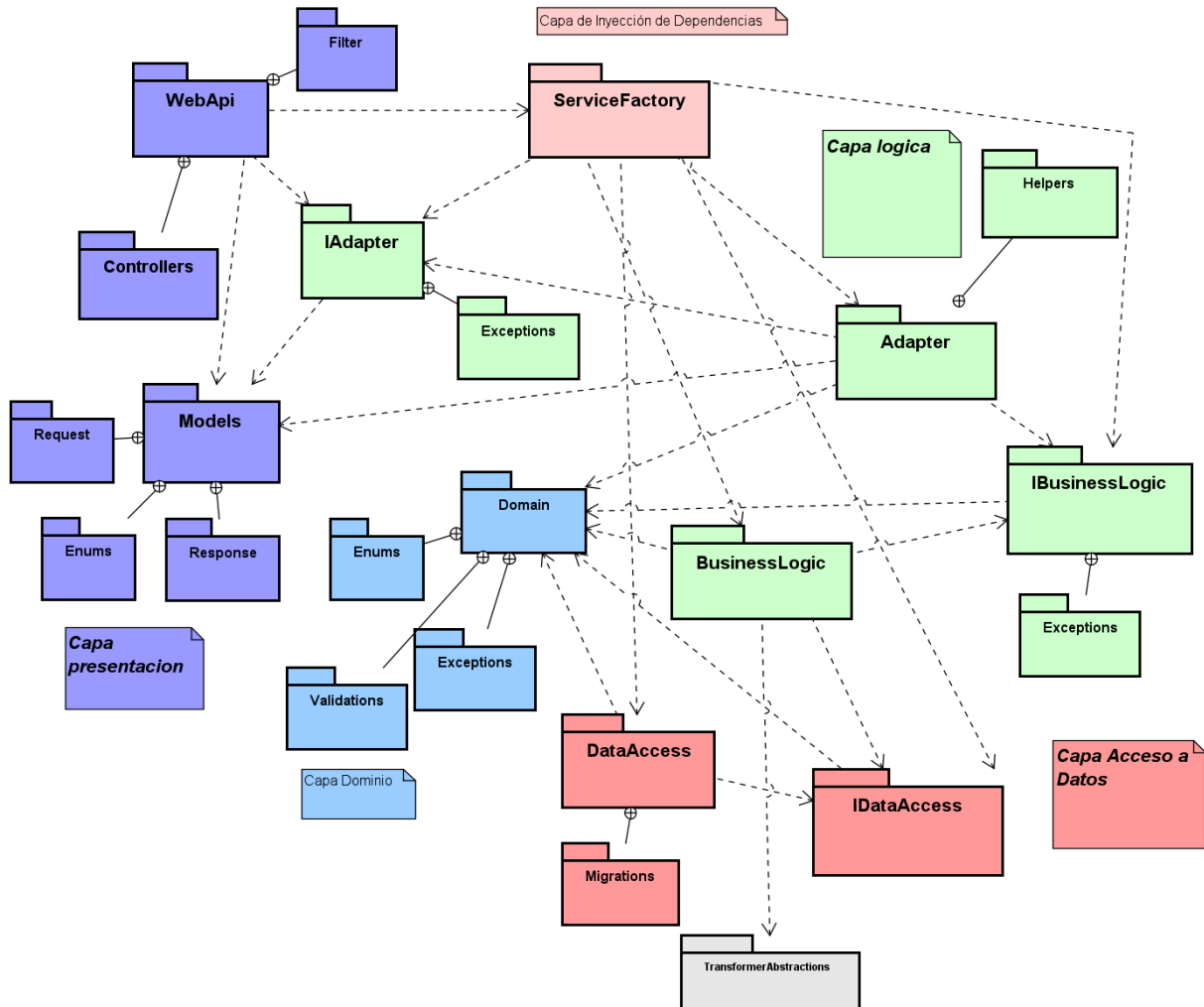
Qué hace la solución

Este proyecto consiste en una API REST que simula conceptos de la programación orientada a objetos. Su objetivo principal es permitir la creación, edición, eliminación y ejecución de clases, atributos, métodos, variables, parámetros y namespaces replicando el comportamiento de un sistema orientado a objetos en C#. La solución también incluye una interfaz frontend desarrollada en Angular.

El sistema soporta diversos aspectos de la programación orientada a objetos, entre ellos: herencia entre clases, Implementación de interfaces, sobrescritura de métodos (virtual / override), modificadores de acceso (public, private, etc.), miembros estáticos e instanciables, clases y métodos abstractos, polimorfismo de parámetros, atributos y variables, implementación de namespaces con herencia, ejecución de métodos con polimorfismo, ejecución de métodos con transformers.

Vista de desarrollo o componentes modelo 4+1

Diagrama general de paquetes



Capa de Presentación:

Esta capa es la encargada de interactuar con el usuario o cliente externo.

`WebApi`: punto de entrada a la aplicación. Expone endpoints que reciben y devuelven datos HTTP.

`Models` (fuera de `WebApi`): contiene las clases `Request`, `Response`, y `Enums` que representan los modelos de datos que viajan entre el cliente y la API. Separarlos permite que otras capas usen estos modelos sin depender directamente de `WebApi`.

Capa Lógica:

Aquí reside la lógica de negocio, es decir, las reglas y operaciones clave del sistema.

`IBusinessLogic` / `BusinessLogic`: interfaz y clase que implementa la lógica del negocio.

`IAdapter` / `Adapter`: patrón Adapter para conectar la capa de presentación con la lógica. Aísla las transformaciones entre modelos DTO y entidades del dominio.

Helpers y Exceptions: utilitarios y manejo de errores comunes dentro de la lógica.

Capa de Dominio (Transversal):

Esta capa representa el corazón del sistema, las entidades y reglas propias del negocio. Aunque es usada por todas las capas, no depende de ninguna.

Domain: clases centrales del negocio (entidades, por ejemplo, SimClass, Method, etc.). Aunque podríamos incluir el dominio dentro de la capa lógica, al separarlo como capa independiente queda más limpio y centrado en las entidades puras del negocio.

Capa de Acceso a Datos

Se encarga de la persistencia, es decir, de acceder a bases de datos u otras fuentes externas.

IDataAccess / DataAccess: interfaz y clase que implementan la lógica de acceso a datos.

Migrations: configuraciones de migración de base de datos.

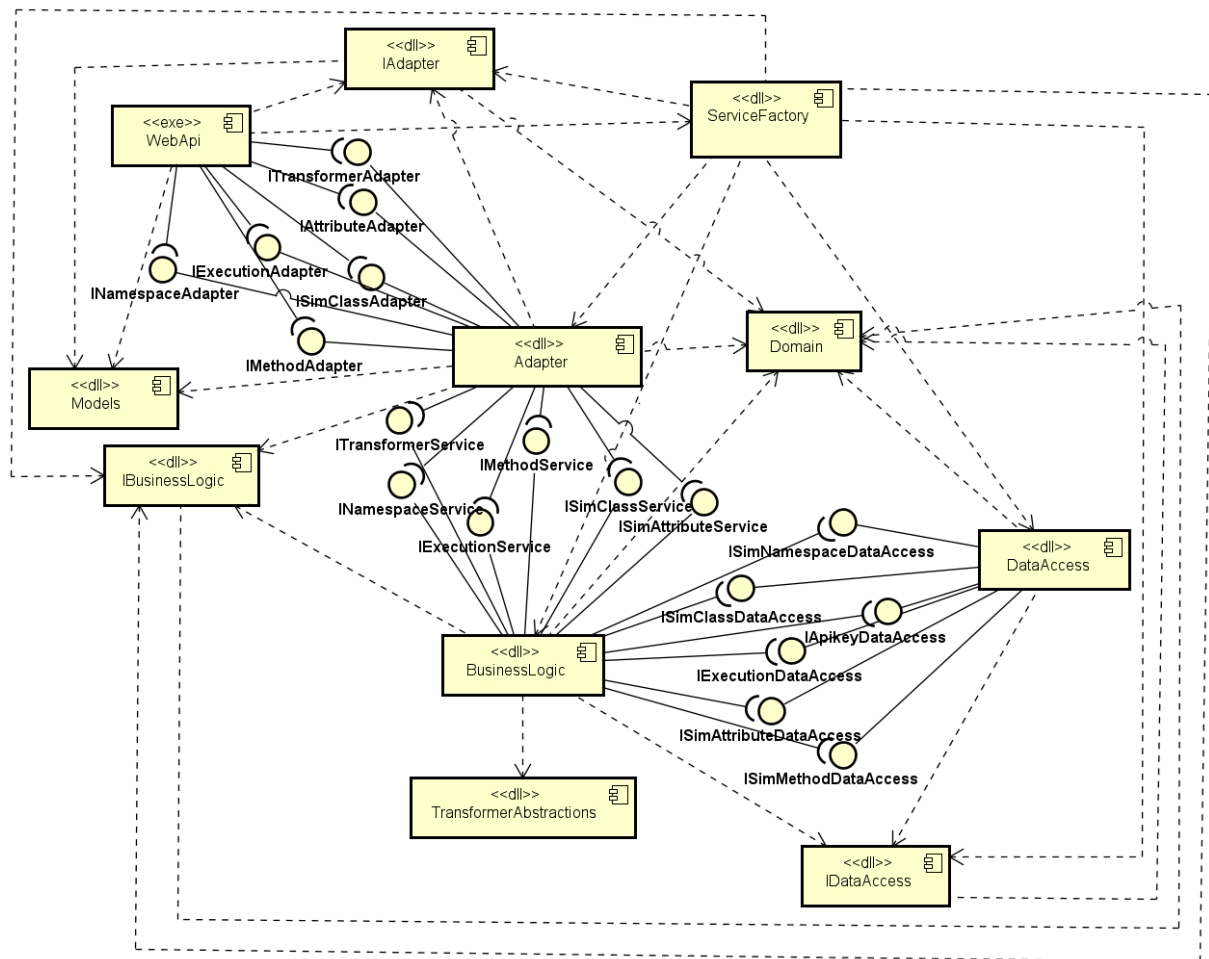
Capa de Inyección de Dependencias (ServiceFactory)

Paquete para inyectar dependencias en la Web Api, desacopla WebApi del resto del sistema.

Breve descripción de responsabilidades de cada paquete

- **WebApi:** Punto de entrada del sistema, expone los endpoints HTTP y maneja las solicitudes del cliente.
- **IAdapter:** Paquete de interfaces que implementa Adapter.
- **Adapter:** El Adapter implementa IAdapter y actúa como puente entre la WebApi y la capa lógica (BusinessLogic), traduciendo datos entre ambos.
- **Domain:** Contiene las entidades centrales del negocio, definiendo las reglas y estructuras fundamentales del sistema.
- **Models:** Define los DTOs para la comunicación de la WebApi.
- **IBusinessLogic:** Paquete de interfaces que implementa BusinessLogic
- **BusinessLogic:** BusinessLogic implementa IBusinessLogic y contiene la lógica del negocio, modelando el comportamiento del sistema según las reglas del dominio.
- **IDataAccess:** Paquete de interfaces que implementa DataAccess.
- **DataAccess:** DataAccess implementa IDataAccess y maneja la interacción con la base de datos, encapsulando la lógica de persistencia.
- **ServiceFactory:** implementa el patrón Factory para la inyección de dependencias en la Web Api, desacoplandola de la capa lógica y de acceso a datos.
- **TransformerAbstractions:** Paquete que define la interfaz IExporter, la cual representa un contrato que deben cumplir los servicios encargados de exportar datos utilizando técnicas de reflection.

Diagrama de componentes



Arquitectura desacoplada: la API usa IAdapter para comunicarse con los adaptadores, que transforman las solicitudes y llaman a la lógica de negocio (IBusinessLogic); esta a su vez delega la persistencia en IDataAccess. Todas las dependencias apuntan a interfaces, cumpliendo los principios de Dependency Inversion y Separation of Concerns.

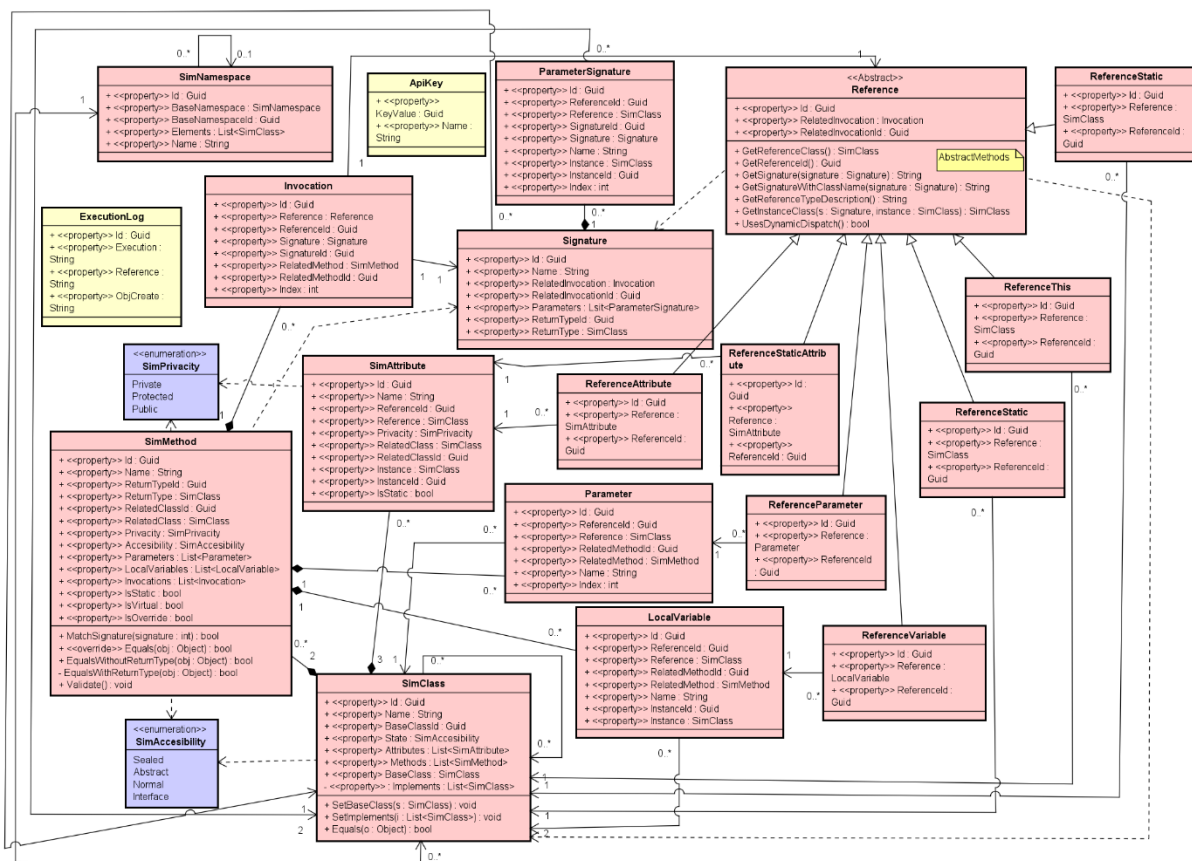
La arquitectura evita ciclos entre paquetes. Las interfaces están ubicadas en paquetes específicos (IAdapter, IDataAccess, etc.) para romper acoplamientos y permitir una navegación limpia de dependencias.

El patrón Factory se utiliza para inyectar dependencias en ServiceFactory, facilitando la construcción de objetos de forma centralizada y controlada, reduciendo acoplamiento de la WebApi del resto de componentes y mejorando la mantenibilidad.

Vista Lógica o Diseño Modelo 4+1

Diagrama de clases de cada paquete

Domain:



En relación con el dominio presentado en la primera entrega, realizamos las siguientes modificaciones. En las clases existentes **SimAttribute**, **LocalVariable** y **ParameterSignature**, reemplazamos el atributo **Type** de tipo **SimClass** por dos referencias: una llamada **Reference**, también de tipo **SimClass**, que indica la clase de referencia; y otra llamada **Instance**, también de tipo **SimClass**, que representa la clase concreta de la instancia. Esta modificación se implementó para dar soporte al polimorfismo.

En el caso de los parámetros (**Parameter**) de un **SimMethod**, su definición se mantiene sin cambios. El polimorfismo se aplica en la clase **Signature** (que contiene **ParameterSignature**), que es la utilizada tanto en las invocaciones como en la ejecución de los métodos.

Agregamos una nueva clase para el requerimiento de namespaces, llamada **SimNamespace** con un **SimNamespace** base y una lista de **SimClass**, también una clase de **ApiKeys** para mapear las keys de la base de datos.

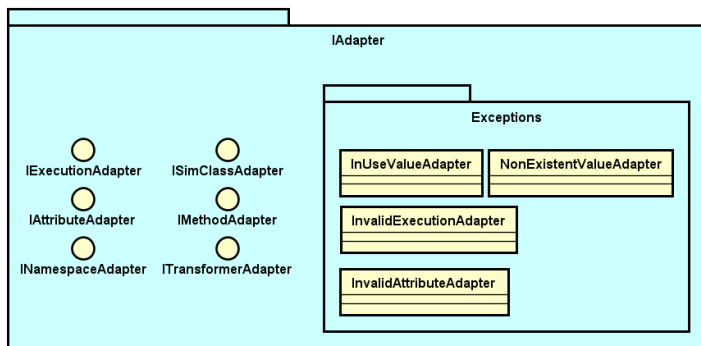
En **Reference** agregamos dos subclases **ReferenceStatic** que modela el comportamiento de una invocación estática, y **ReferenceStaticAttribute** para los atributos estáticos que tienen diferente comportamiento al de los atributos no estáticos, ahora las referencias tienen un método que retorna la **SimClass** de la referencia y otro que retorna la instancia, ese nuevo método **GetInstanceClass**

recibe por parámetro una SimClass debido a que en el caso de ReferenceThis se necesita la instancia desde donde se ejecuta el método, también se le pasa una Signature la cual se utiliza para el caso de los parámetros, ya que ReferenceParameter tiene relacionado un parámetro, pero necesitamos ver la instancia de ese parámetro (polimorfismo), entonces al pasar la Signature en la ejecución podemos ver con que tipo polimórfico se llama al parámetro.

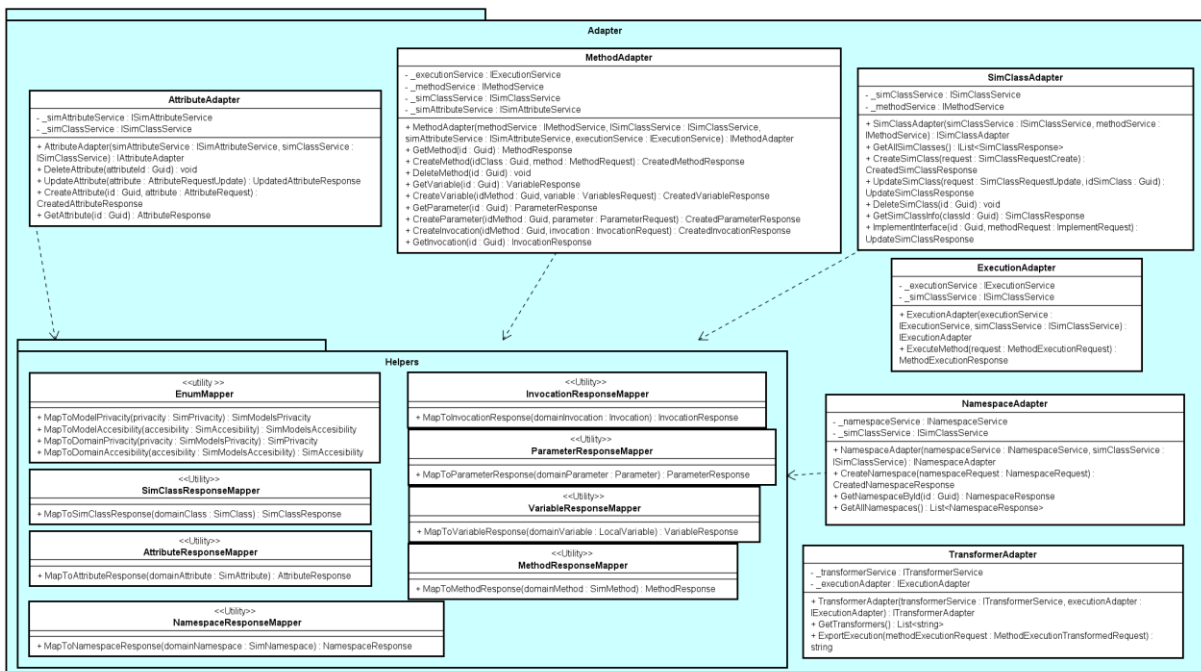
Para las interfaces agregamos un nuevo tipo en el Enum SimAccessibility llamado interface, y le agregamos una lista SimClass de las clases que una clase implementa, este enum también lo usa un método (lo usamos para reflejar que si un método es tipo interface no puede tener invocaciones, variables etc).

También agregamos los atributos IsStatic, IsVirtual, IsOverride en los métodos para modelar dichas propiedades, y en atributos agregamos IsStatic.

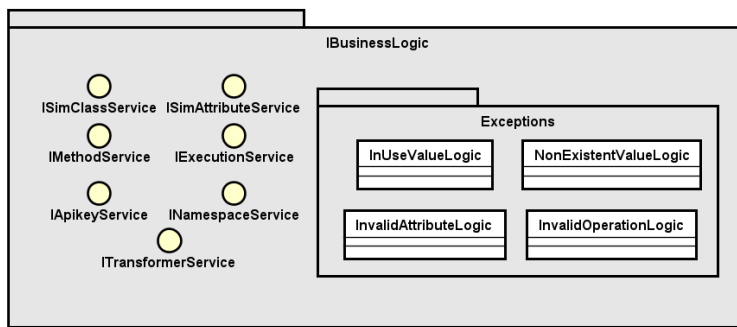
IAdapter:



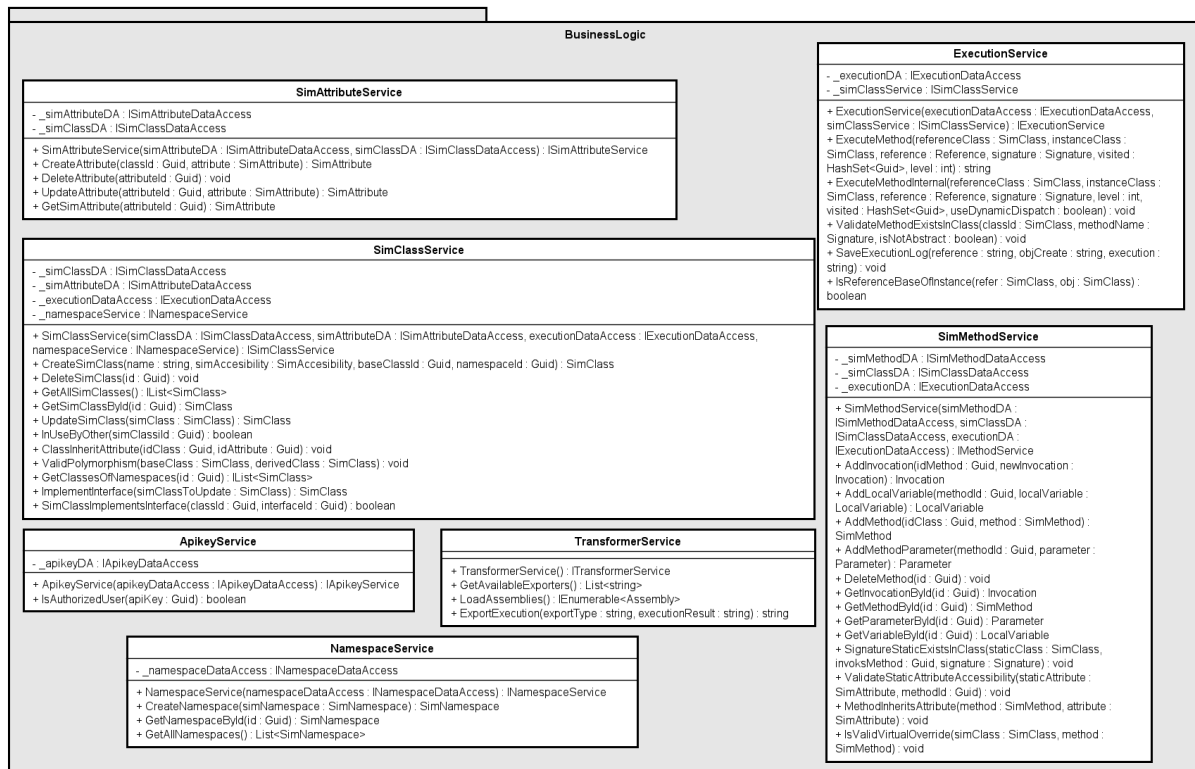
Adapter:



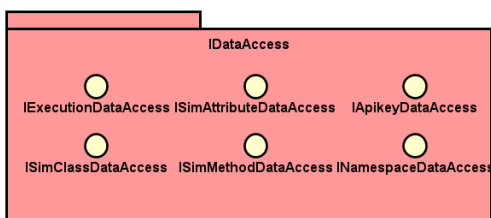
IBusinessLogic:



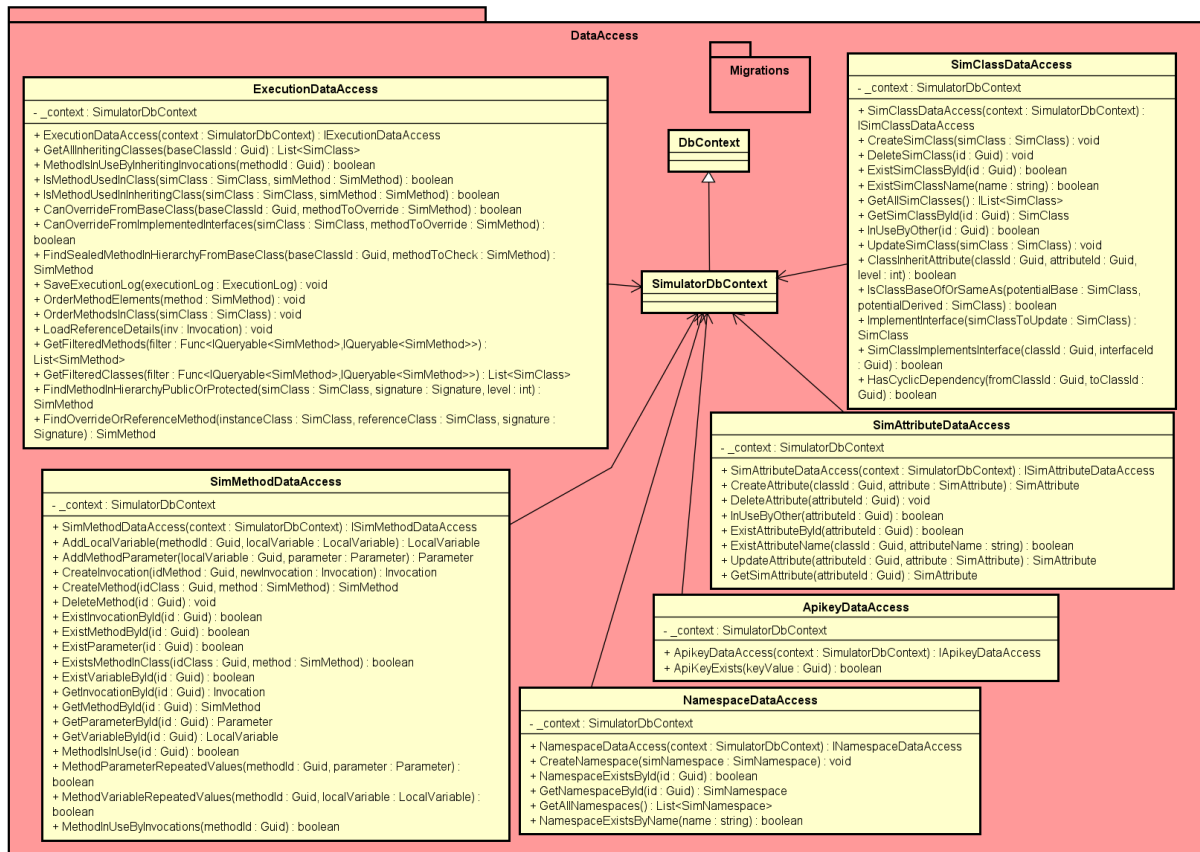
BusinessLogic:



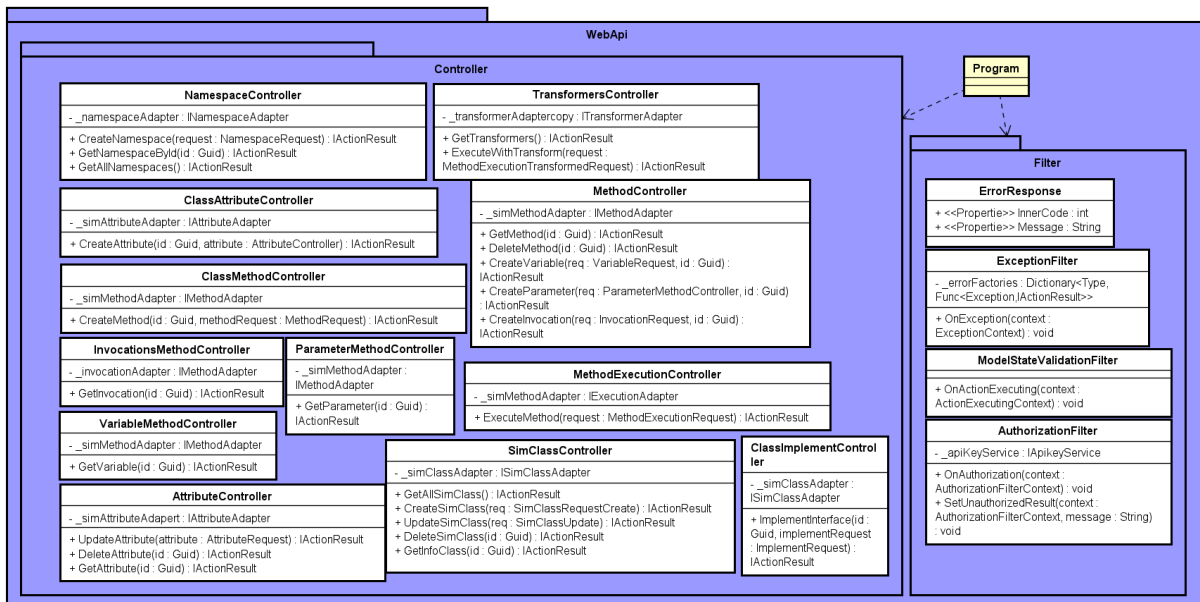
IDataAcces:



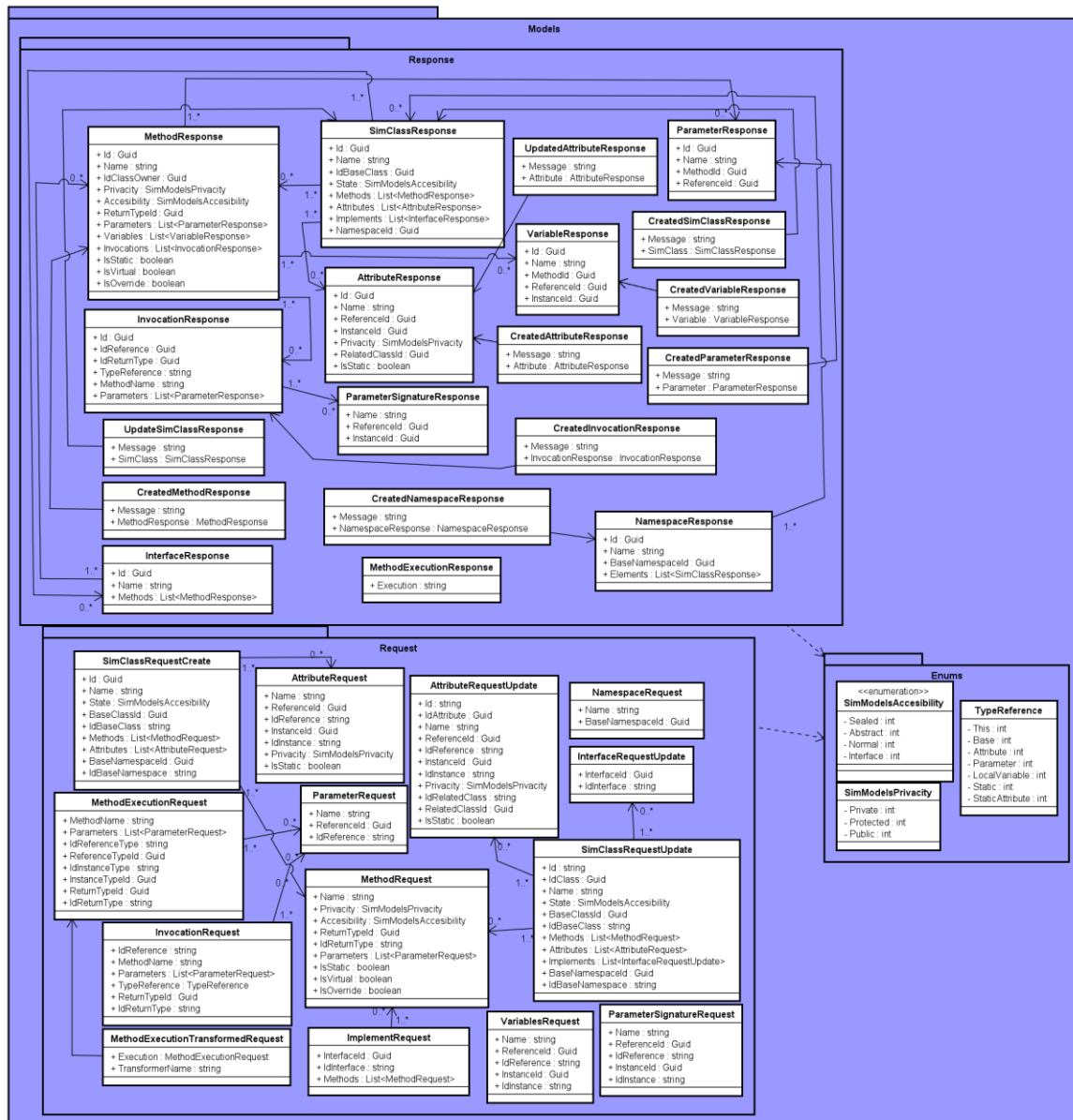
DataAccess:



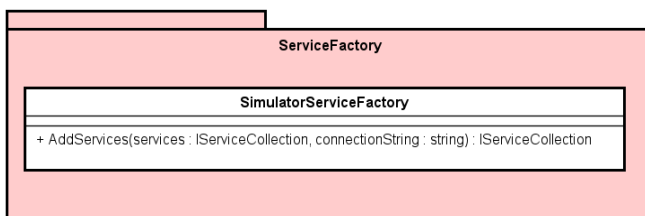
WebApi



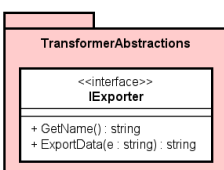
Models:



ServiceFactory:



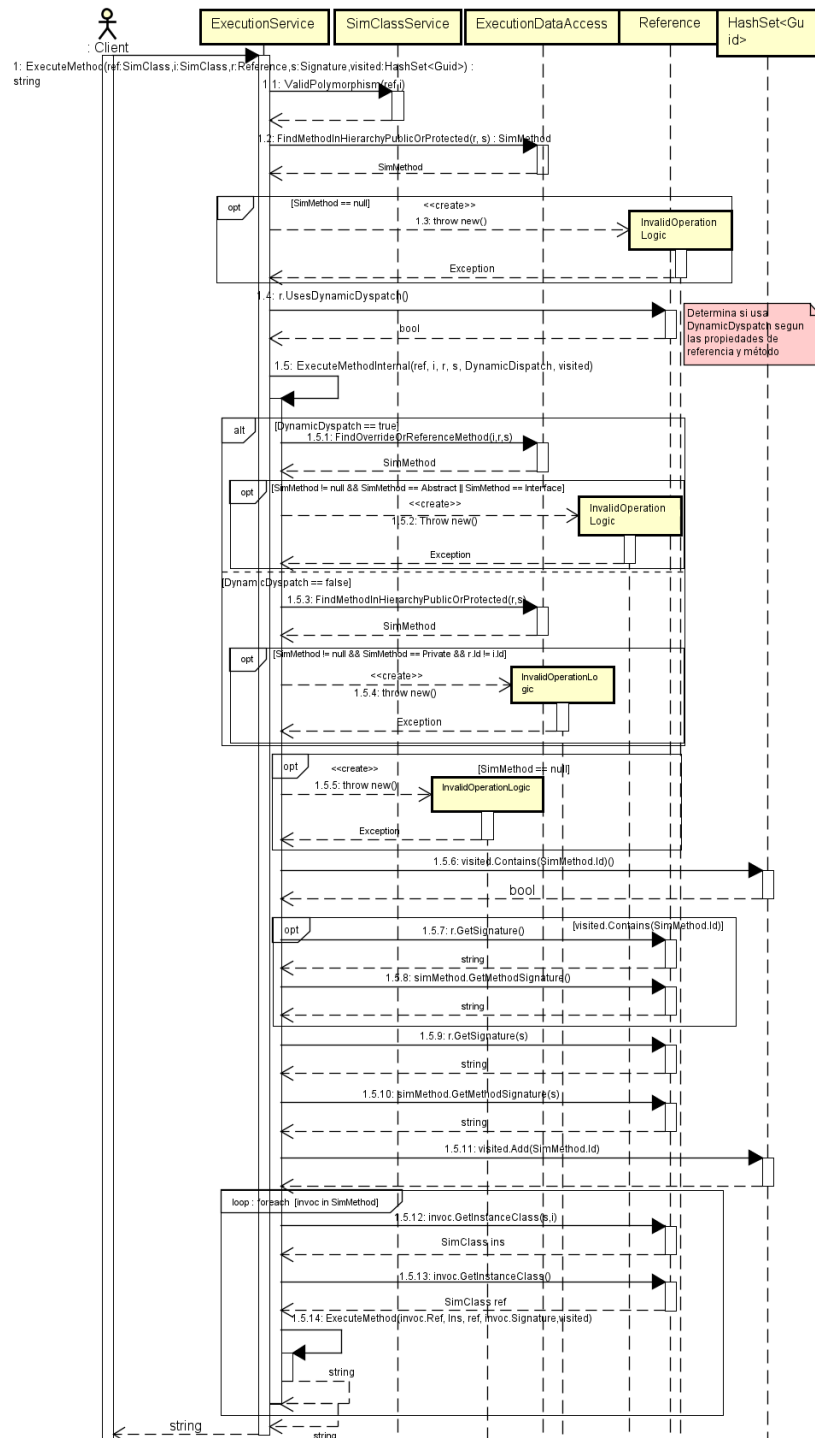
TransformerAbstractions:



Funcionalidades de mayor relevancia

Diagramas de interacción que muestran las clases involucradas

Ejecutar un método:



Para ejecutar el método recursivo `ExecuteMethod`, se le pasan como parámetros dos objetos `SimClass` (uno que representa la clase de referencia y otro la de instancia) una `Signature` y una referencia. Esta `Signature` está compuesta por un nombre, una lista de `ParameterSignature` (que permite polimorfismo) y un `ReturnType` de tipo `SimClass`.

El proceso comienza buscando el método en la jerarquía de la clase de referencia, emulando el comportamiento de C#. Luego, se determina si se utilizará enlace dinámico (dynamic dispatch) o enlace estático. Esta decisión se basa en:

La propiedad `DynamicDispatch` de la `Reference`. Por ejemplo, `ReferenceBase` o `ReferenceStatic` tienen esta propiedad en `false`, por lo que no permiten enlace dinámico.

Además, incluso si la `Reference` indica `true`, solo se aplicará enlace dinámico si el método está marcado como `virtual`, `abstract` o pertenece a una interfaz. Si el método no es `virtual`, se usará enlace estático, aunque la referencia lo permita.

Una vez determinado el tipo de enlace:

Si se utiliza enlace dinámico, se buscará en la jerarquía desde la clase de instancia hacia la clase de referencia un método que sobrescriba (`override`) la firma buscada. Si se encuentra, se ejecutará ese método. Si no, se ejecutará el método definido en la clase de referencia.

Si se utiliza enlace estático, se ejecutará directamente el método encontrado en la clase de referencia.

Una vez determinado el método a ejecutar mediante los mecanismos de enlace, se obtiene su lista de `Invocations`. Para cada invocación, se extraen la referencia, la instancia, la clase referencia y la `Signature` correspondientes, y se vuelve a invocar recursivamente `ExecuteMethod`, repitiendo el proceso para cada una en un `loop`.

El método `FindMethodInHierarchyPublicOrProtected` busca recursivamente un método a partir de una `Signature` y una clase. Si no lo encuentra en la clase actual, continúa la búsqueda en su clase base, y así sucesivamente hasta que no haya más clases en la jerarquía. Si el método es `null` o no tiene propiedades para ser ejecutado (`abstract` o `interface`) tiramos excepción. También hay una función que valida el polimorfismo cuidando de que la referencia sea base tanto como clase base o implementación de la instancia.

Otra funcionalidad clave que modelamos en este método es la posibilidad de que el usuario ejecute un método eligiendo el polimorfismo deseado en sus parámetros. Esto se implementa en la clase `ReferenceParameter`, cuando se necesita obtener la clase de instancia de un parámetro, se le pasa la `Signature` completa (que contiene una lista de `ParameterSignature`). Cada `Parameter` contiene su posición (índice) dentro de la lista de parámetros. A partir de ese índice, se accede a la posición correspondiente en la lista de `ParameterSignature` y se obtiene la clase de instancia asociada, permitiendo así que el usuario defina dinámicamente qué tipo utilizar en cada parámetro, respetando el polimorfismo.

Descripción de jerarquías de herencia utilizadas

En este obligatorio se mantuvo la jerarquía de herencia utilizada en la entrega anterior, en la clase `"Reference"`, implementando una clase abstracta y 7 subclases concretas: `ReferenceThis`, `ReferenceBase`, `ReferenceVariable`, `ReferenceParameter` y `ReferenceAttribute`, se le agrego `ReferenceStatic` y `ReferenceStaticAttribute`, que modela el comportamiento de las invocaciones. Esta herencia nos permitió que el método encargado de ejecutar métodos fuera más mantenible y legible,

además de facilitar la extensión de nuevos comportamientos requeridos, como la ejecución de métodos estáticos y la ejecución de métodos definidos en atributos estáticos, sin necesidad de modificar el código existente. Esto se alinea con el principio Open/Closed, que indica que el software debe estar abierto a la extensión, pero cerrado a la modificación.

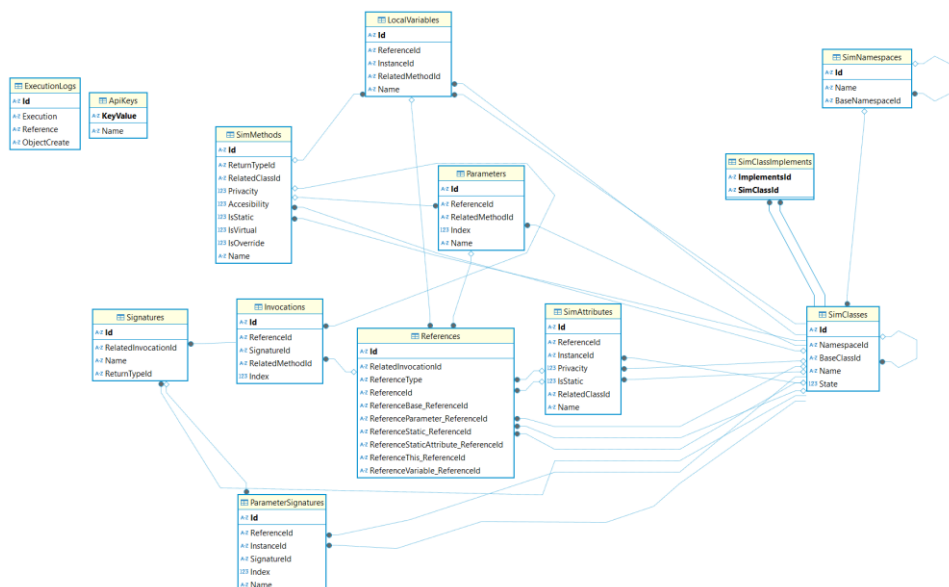
Gracias a esta estructura, fue posible incorporar nuevas funcionalidades, como el modelado del enlace dinámico de cada referencia (cuando corresponde), sin recurrir a estructuras como switch, que hubieran afectado negativamente la legibilidad de un algoritmo ya complejo.

Además, la herencia facilitó la implementación de polimorfismo en los parámetros, tal como requería el cliente. En particular, la clase ReferenceParameter recibe una firma (Signature) con una ParameterSignature, la cual define cómo debe obtenerse la instancia del parámetro mediante el método GetInstance(SimClass, Signature). Sin esta arquitectura, hubiera sido muy difícil implementar el polimorfismo de parámetros, dado que un parámetro puede ser invocado desde múltiples métodos con combinaciones polimórficas distintas, y por tanto no posee por sí solo ni una instancia ni una referencia concreta.

Se realizaron ajustes menores, como el hecho de que el método GetInstance ahora también recibe un objeto SimClass, que representa la instancia desde la cual se está ejecutando el método. Esto es necesario para resolver correctamente el this (ReferenceThis) en tiempo de ejecución, ya que su valor depende del contexto desde el cual se realiza la invocación.

También decidimos incorporar una nueva clase llamada ReferenceStaticAttribute, separándola de ReferenceAttribute, ya que consideramos que ambos casos representan comportamientos distintos.

Modelo de tablas de la estructura de la base de datos



Descripción del mecanismo de acceso a datos

Para el acceso a datos utilizamos la clase DbContext de Entity Framework Core de la cual hereda SimulatorDbContext, a través de un conjunto de repositorios que encapsulan las operaciones básicas como agregar, eliminar, modificar y consultar entidades. Cada repositorio implementa una interfaz

correspondiente, lo que nos permite definir contratos claros y desacoplar la lógica de acceso a datos de la lógica de negocio. Se creó un repositorio en la carpeta DataAccess por funcionalidad principal, y cada uno fue inyectado en los servicios correspondientes mediante inyección de dependencias. Este enfoque promueve una arquitectura modular y reutilizable.

Eager Loading y consultas complejas:

Para mapear entidades utilizamos Eager Loading, ya que gran parte de las consultas requieren cargar múltiples niveles de navegación. Por ejemplo:

Comparar dos métodos (SimMethod) implica verificar si coinciden sus parámetros (Parameters) e invocaciones (Invocations), manteniendo el orden definido.

Determinar si un método o atributo es heredado exige navegar recursivamente por la jerarquía de clases (BaseClass).

Consultas sobre herencia también incluyen verificar qué clases heredan de una determinada clase.

Dado que estas consultas involucran objetos anidados y múltiples niveles de navegación, fue necesario cargar en memoria todas las propiedades relacionadas mediante Eager Loading.

Además, debido a que el orden de parámetros e invocaciones es relevante para la lógica del simulador, cada entidad (Parameter, ParameterSignature, Invocation) posee un campo Index, que indica su posición en la colección correspondiente. Esta estrategia se utilizó porque, al persistir los datos, Entity Framework los ordena por defecto por identificador (Id Guid), lo que no refleja el orden lógico deseado.

Para modelar la jerarquía de clases de Reference, se utilizó el enfoque TPH (Table per Hierarchy), priorizando evitar la complejidad de múltiples joins a cambio de permitir la existencia de campos nulos en la tabla.

En cuanto a la autorrelación de muchos a muchos en SimClass para representar las implementaciones, se utilizó una tabla intermedia explícita denominada SimClassImplements (explicit intersection table mapping).

Justificación del diseño

Mecanismos de inyección de dependencias

Se mantuvo el mismo enfoque que en la entrega anterior, utilizando ServiceFactory para registrar las dependencias mediante AddScoped, lo que garantiza una nueva instancia por solicitud. Se aplicó la inyección por constructor, permitiendo que las clases dependan de interfaces en lugar de implementaciones concretas. Este diseño favorece el desacoplamiento, mejora la mantenibilidad del código y facilita la realización de pruebas unitarias.

Principios SOLID

Single Responsibility Principle (SRP): aplicamos este principio mediante una clara separación por capas, donde cada capa y namespace contiene clases con responsabilidades bien definidas. Cada clase se enfoca exclusivamente en representar una única razón de ser, a excepción de las clases del

paquete de Adapter, el cual estaba destinado exclusivamente a transformar los servicios en Models para servir a la WebApi y viceversa escondiendo la complejidad de los DTO de Models a BussinesLogic, pero no se logró, ya que estos tienen algo de lógica.

Open/Closed Principle (OCP): Usamos el principio Open/Closed en la clase Reference porque queríamos permitir agregar nuevas formas de representar desde dónde se hace una invocación de método sin tener que modificar el código existente. En lugar de hacer una gran estructura condicional (como un if o switch) en una sola clase para manejar todas las formas posibles (this, base, atributos, parámetros, variables, atributos estáticos, static), creamos una clase abstracta Reference con métodos como GetReferenceClass(), y luego definimos una subclase distinta para cada caso.

Esto nos permitió extender el comportamiento (agregando una nueva subclase como ReferenceStatic, ReferenceStaticAttribute), sin modificar el código de las subclases anteriores ni de Reference.

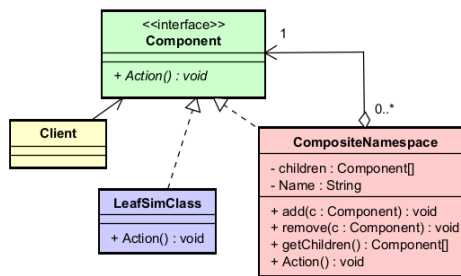
Liskov Substitution Principle (LSP): seguimos este principio al diseñar nuestras interfaces y clases de forma que las implementaciones concretas puedan sustituir a sus abstracciones sin alterar el comportamiento esperado del sistema. Por ejemplo, los servicios y componentes que dependen de interfaces como ISimClassDataAccess o ISimClassService pueden funcionar correctamente sin importar qué implementación concreta se inyecte. Esto asegura que las clases hijas o derivadas respeten el contrato definido por sus clases base o interfaces.

Interface Segregation Principle (ISP): se procuró aplicar este principio evitando dependencias innecesarias. Sin embargo, debido a la complejidad de entidades como SimClassService, que requiere acceso a atributos, ejecuciones y namespaces, se utilizaron interfaces amplias (como ISimAttributeDataAccess o INamespaceService) sin emplear todos sus métodos. En los adapters ocurrió algo similar: para generar una salida lo más client-friendly posible, fue necesario reunir información proveniente de distintas entidades relacionadas, lo que llevó a depender de múltiples interfaces con funcionalidades distribuidas. Por razones de tiempo, no fue posible refactorizar estas interfaces, generando ciertos acoplamientos que podrían mejorarse con una mejor segregación.

Dependency Inversion Principle (DIP): tal como se mencionó anteriormente, este principio se aplicó de forma consistente a lo largo del proyecto para lograr un diseño desacoplado. Las clases de alto nivel, como las de la capa de BusinessLogic ejemplo (SimClassService), no dependen directamente de implementaciones concretas (por ejemplo, SimClassDataAccess), sino de interfaces (como ISimClassDataAccess). Esto nos permite cambiar o extender las implementaciones sin modificar la lógica de negocio.

Otros principios de diseño

Composite: Se evaluó la implementación del patrón composite para el requerimiento de los namespaces, llegamos a la conclusión de que tendríamos que reducir el dominio a este uml (en este uml faltan todos los atributos que tiene una SimClass las cuales se pueden apreciar en el UML de dominio, para simplificar la visualización). Como una SimClass contiene todo lo que tiene el dominio del problema (atributos, métodos que a su vez estos contienen las invocaciones parámetros etc). Esta todo encapsulado en una SimClass, por lo que sería la hoja del árbol, y estas hojas pueden estar dentro de namespaces (Composites) los cuales las contienen y pueden contener otros namespaces (sus namespaces derivados).



A la conclusión que llegamos es que implementar esto en nuestro proyecto no nos traería ningún beneficio, ya que según los requerimientos extraídos del foro un namespace puede tener simClasses con tipos de otros namespaces (Ej: Referencias, instancias, retornos de métodos, puede usar hasta métodos de otros namespaces en sus invocaciones), por lo que implementar este patrón significaría una violación del principio de encapsulamiento: las SimClasses dentro de un namespace pueden referenciar clases, métodos e instancias de otros namespaces. Esto rompe la idea de jerarquía de composición cerrada que da sentido al patrón Composite. No hay acciones comunes significativas entre Namespace y SimClass que justifiquen una interfaz compartida, como se ve en este UML no vemos métodos en común "Action()". El uso del patrón complicaría la lógica de navegación y las relaciones en la base de datos.

Fábrica: Aplicamos este patrón en ServiceFactory esta clase actúa como una fábrica de configuración de servicios, centralizando el registro de todas las dependencias necesarias para la aplicación. A través del método AddServices, se registran en el contenedor de dependencias las implementaciones de los adapters, servicios de negocio y acceso a datos, cada uno con su respectiva interfaz. Esto permite que los controladores de la Web API reciban las dependencias necesarias sin conocer sus implementaciones concretas, promoviendo el desacoplamiento y la inyección de dependencias. Además, se configura el contexto de base de datos SimulatorDbContext usando Entity Framework Core con una conexión SQL Server.

Repositorio: utilizamos este patrón para separar la lógica de acceso a la base de datos del resto del sistema, promoviendo la cohesión y el desacoplamiento. Definimos una interfaz IDataAccess que expone operaciones CRUD (crear, leer, actualizar y borrar), y su implementación concreta DataAccess, que interactúa directamente con la base de datos. Además de las operaciones básicas, incluimos consultas más complejas pensamos que mejoraríamos el rendimiento de nuestra aplicación con ello.

Adapter: Se intento implementar este patrón para desacoplar los models de la lógica pero no se cumplió al 100%, en algunas clases mas que llamar a los servicios parece que se validan cosas también en dicha capa.

Decisiones propias de diseño

- No se puede agregar parámetros por medio del endpoint agregar parámetros, a los métodos de tipo interface, abstract ni métodos virtuales, esto para evitar problemas de cambio de firmas al realizar override (al cambiar parámetros hay desreferencias), solo se puede agregar los parámetros al crear el método.

- Una interfaz no hereda de nada ni implementa ninguna interfaz, aunque nosotros le pusimos por defecto que hereda por object, pero al crearla, o no se pone nada o se pone object como clase base.

- Si un método no es virtual no puede ser override ni viceversa, esto lo asumimos así, aunque está implícito en C#, para facilitarnos el algoritmo de ejecución de métodos.
- Para evitar incongruencias si un método es virtual y no override, interface o abstract no se puede borrar por medio del endpoint de borrar metodo ya que, si otro método de una clase inferior le está haciendo override, al borrarlo quedaría un override sin una clase virtual superior lo que es un error de compilación en C#, estos se pueden borrar al borrar su clase si no está en uso.
- No se puede agregar métodos a interfaces o clases abstractas en uso para evitar “cambios de contrato” en sus subclases.
- El api key que se encuentra en la base de datos esta harcodeada en el frontend como así lo piden en el foro, al cargar la aplicación de angular se carga una key al iniciar la aplicación esta se guarda en el local storage si se borra el local storage y no se reinicia la aplicación va a dar error forbbiden al ingresar a la página de los transformers.
- En la implementación actual se valida que el nombre de cada clase sea único entre todos los namespaces. Esta validación se incorporó para evitar ambigüedades al referenciar clases, ya que el sistema no implementa el concepto de using, como se especificó en una intervención en el foro.
- En la implementación actual, se valida que el nombre de cada Namespace sea único en toda la base de datos, sin considerar su nivel jerárquico.

Métricas principios a nivel de paquetes

Projects	# Types	# Abstract Types	Afferent Coupling	Efferent Coupling	Relational Cohesion	Instability	Abstractness	Distance
Domain	24	1	38	28	3.54	0.42	0.04	0.38
IBusinessLogic	11	7	15	23	0.09	0.61	0.64	0.17
Models	36	0	35	22	1.11	0.39	0	0.43
IAadapter	10	6	20	37	0.1	0.65	0.6	0.18
Adapter	17	0	1	100	1.12	0.99	0	0.01
IDataAccess	6	6	13	21	0.17	0.62	1	0.44
TransformerAbstractions	1	1	1	2	1	0.67	1	0.47
BusinessLogic	7	0	1	85	0.14	0.99	0	0.01
DataAccess	28	0	1	128	1.21	0.99	0	0.01
ServiceFactory	1	0	1	63	1	0.98	0	0.01
WebApi	21	0	0	140	0.71	1	0	0

ADP: Se cumple ya que .NET no permite ciclos en paquetes.

REP: Principio de equivalencia reuso/liberacion a nivel de paquete, este principio no lo cumplimos ya que establece que lo ideal sería por paquete un ensamblado, y nuestros paquetes tienen sub-paquetes porque favorecimos la mantenibilidad antes que la reusabilidad.

CCP: Principio de Clausura común, establece que las clases que cambian por las mismas razones y al mismo tiempo deben estar agrupadas en el mismo paquete.

Este principio no pudimos cumplirlo ya que como nuestro sistema está comprendido en capas y cada capa cuenta con una responsabilidad el impacto en uno impacta en los otros paquetes ya que por ejemplo si quiero agregar una funcionalidad, voy a tener que cambiar el paquete dominio, pero

también el de DataAccess ya que voy a tener que adaptar la base de datos a ese nuevo cambio, también cambiaría la capa lógica BussinesLogic teniendo que incorporar nuevas validaciones.

Pero si lo vemos desde el punto de vista de infraestructura se cumple porque si quiero cambiar una validación en la logica, por ejemplo, que se pueda repetir el mismo nombre en una clase, lo cambio en la lógica y no impacta en otro lado.

CRP :

El principio de reúso común establece que las clases que tienden a ser utilizadas juntas deben agruparse en el mismo paquete. En nuestro diseño, este principio se cumple en paquetes como Domain y Models, ya que las clases dentro de ellos se utilizan en conjunto de forma frecuente. Por ejemplo, cuando DataAccess utiliza clases del paquete Domain, tiende a requerir a todas ellas, y lo mismo ocurre con Adapter al usar las clases de Models.

Una muestra de esto es en la métrica de Relational Cohesion de Domain 3.54 que es la más alta, esto indica que cuando se usa ese paquete se usan todas las clases ya que están todas fuertemente cohesionadas entre sí.

Sin embargo, en otros casos como BusinessLogic y DataAccess, nos resultó difícil aplicar completamente este principio. Esto se debe a que SimClass encapsula todo el dominio (atributos, métodos, variables, parámetros), y muchos métodos requerían acceder a esta clase. Por ejemplo, para validar un método era necesario invocar el servicio de SimClassService, también para realizar una ejecución de un método, o crear un parametro dentro de un metodo, todo requería servicios de una SimClassService, casi todas las operaciones estaban relacionadas entre sí dentro del dominio del problema. Esta fuerte interdependencia dificultó la separación de responsabilidades y, por ende, el cumplimiento total del CRP en esas capas.

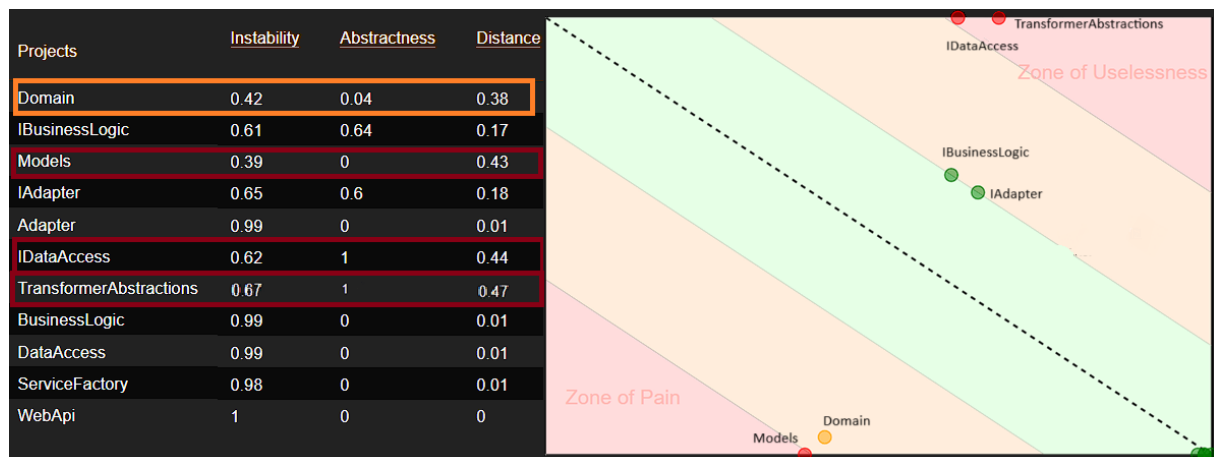
SDP: El principio de dependencias estables establece que los módulos deben depender de paquetes más estables que ellos mismos.

Como podemos apreciar en la imagen Domain, Models, TransformerAbstraction no dependen de nadie por lo que cumplen SDP, IBussinesLogic, IDataAccess, IAdapter cumplen porque dependen de Domain 0.42 y Models 0.39 que son menos inestables, WebApi con maxima inestabilidad depende de IAdapter 0.65, ServiceFactory 0.98 y Models con 0.39 por lo que cumple SDP, ServiceFactory depende de casi todos los paquetes y como tiene la maxima inestabilidad cumple SDP porque son igual o menor en inestabilidad los otros paquetes, DataAccess 0.99 cumple porque depende de IDataAccess 0.62 y Domain 0.42 los cuales tienen inestabilidad menor, BusinessLogic 0.99 cumple porque depende de IDataAccess 0.62 y Domain 0.42 con inestabilidad menor, también tiene dependencia con TransformerAbstraction para reflection 0.89, Adapter también cumple porque tiene máxima inestabilidad y depende de IAdapter con 0.65, Models 0.39 y Domain con 0.42.

Por lo tanto, nuestro proyecto cumple con SDP.

Projects	Instability
Domain	0.42
IBusinessLogic	0.61
Models	0.39
IAdapter	0.65
Adapter	0.99
IDataAccess	0.62
TransformerAbstractions	0.67
BusinessLogic	0.99
DataAccess	0.99
ServiceFactory	0.98
WebApi	1

SAP: Principio de Abstracciones Estables establece que los paquetes estables deben ser abstractos, mientras que los paquetes inestables deben ser concretos.



Como podemos apreciar en el grafico todos los paquetes menos el de Domain , Models , IDataAccess y TransformerAbstractions cumplen este principio ya q el resto de estos se encuentra en un nivel adecuado de aceptación $0.75 < A+I < 1.25$ cercano a 1 en zona verde.

Para el caso de los que se encuentran en zonas rojas:

Zone of Pain, zona de dolor:

Los paquetes Domain y Models tienen baja abstracción (0 y 0.04 respectivamente), pero también baja inestabilidad (0.39 y 0.42). Es decir, son estables pero concretos, lo que los vuelve rígidos: muchos otros paquetes dependen de ellos, pero al ser concretos, son difíciles de extender sin afectar a los clientes. Esto los posiciona en la "zona del dolor", donde las modificaciones pueden generar efectos colaterales. Esta situación se justifica en nuestro caso porque el paquete Domain concentra toda la lógica estructural del negocio: clases como SimClass, atributos, métodos, parámetros, etc. Como define la base del modelo, muchos paquetes dependen de él, pero él no depende de ninguno, lo que le otorga esa estabilidad. Sin embargo, al estar compuesto mayormente por clases concretas y no por abstracciones, su capacidad de extensión es limitada sin afectar a los clientes.

El mismo razonamiento aplica a Models, que contiene clases utilizadas directamente por capas como Adapter, WebApi como DTO, pero que tampoco abstrae su contenido.

Zone of Uselessness, zona de inutilidad:

El paquete IDataAccess tiene alta inestabilidad (0.62) y abstracción máxima (1.0). Esto lo ubica en la "zona de inutilidad". IDataAccess, pese a ser 100% abstracto, alcanza alta inestabilidad (0.62) debido a numerosas dependencias salientes hacia tipos de dominio, múltiples firmas de métodos específicas, parámetros de consulta complejos y retornos genéricos, contradiciendo el propósito estabilizador de las abstracciones.

El paquete TransformerAbstractions se encuentra en la zona de inutilidad únicamente por su inestabilidad alta, ya que es totalmente abstracto ($A = 1$) pero no depende de ningún paquete, mientras que otros paquetes sí dependen de él, y esto es natural ya que solo es un contrato que definimos para implementar reflection.

Mejoras en las métricas de paquetes

Se logró una mejora en el cumplimiento del principio SAP en los paquetes IBusinessLogic e IAdapter al mover las excepciones que estaban en BusinessLogic y Adapter respectivamente hacia sus paquetes de interfaces (IBusinessLogic e IAdapter). Este cambio disminuyó la métrica de abstracción de los paquetes de interfaces, haciendo que la métrica de abstracción se acerque al valor de inestabilidad, alineándose mejor con la diagonal ideal del gráfico SAP. Además, este ajuste permitió eliminar dependencias innecesarias, WebApi ya no depende del paquete Adapter, ya que su filtro de excepciones ahora utiliza directamente las excepciones definidas en IAdapter. De forma similar, Adapter dejó de depender de las excepciones ubicadas en BusinessLogic, utilizándolas ahora desde IBusinessLogic.

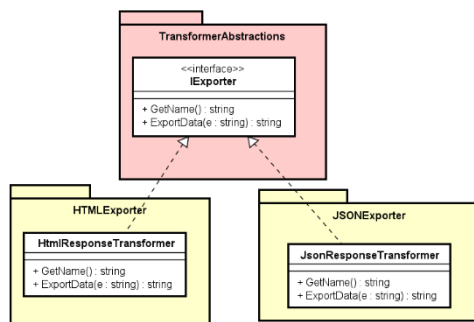
Con esto, se logró una arquitectura más desacoplada y coherente con los principios de diseño.

Descripción del manejo de excepciones

Se mantuvo el uso de excepciones personalizadas para representar errores del dominio, mapeadas a códigos HTTP e InnerCode. Estas se capturan mediante un exception filter global en la Webapi, el cual solo maneja excepciones del paquete IAdapter. Las excepciones provenientes de Domain y BusinessLogic se transforman en Adapter, manteniendo así el desacoplamiento de la Webapi respecto al resto de los paquetes. Además, con respecto a la entrega anterior, migramos las excepciones de Adapter y BusinessLogic hacia sus correspondientes interfaces teniendo en cuenta las devoluciones de los docentes e intentando desacoplar.

Implementación de un Nuevo Transformador Reflection

La interfaz IExporter en el paquete TransformerAbstractions define el contrato para todos los exportadores que quieren ofrecer servicios de transformación de datos en el sistema. Como la ejecución de métodos retorna un string, no vimos la necesidad de crear una clase auxiliar para modelar.



Guía de Implementación: Interfaz IExporter

La interfaz IExporter en el paquete TransformerAbstractions define el contrato para todos los exportadores que quieren ofrecer servicios de transformación de datos en el sistema:

```
public interface IExporter
{
    string ExportData(string source);
}
```

```
    string GetName();  
}
```

La interfaz requiere implementar dos métodos simples:

El metodo **string ExportData(string source)** que tiene como propósito transformar los datos de ejecución al formato deseado retorna los datos transformados en el formato específico del exportador.

El metodo **string GetName()** tiene el propósito de proporcionar el nombre del exportador que aparecerá en la interfaz retorna un identificador único como "HTML", "JSON", etc.

Pasos para Crear un Nuevo Exportador

Crear un Proyecto de Biblioteca de Clases:

```
dotnet new classlib -n MiExportador
```

Añadir Referencia a TransformerAbstractions.dll (un dll compilado del paquete TransformerAbstractions):

Copiar TransformerAbstractions.dll en la carpeta del proyecto

```
Actualizar el archivo de proyecto (.csproj):<ItemGroup>  
<Reference Include="TransformerAbstractions">  
  <HintPath>TransformerAbstractions.dll</HintPath>  
</Reference>  
</ItemGroup>
```

Implementar la Clase Exportadora:using TransformerAbstractions;
namespace MiExportador;

```
public class MiFormatoTransformer : IExporter  
{  
    public string GetName() => "MiFormato";  
  
    public string ExportData(string executionResult)  
    {  
        Lógica de transformación personalizada aquí  
        return $"<mi-formato>{executionResult}</mi-formato>";  
    }  
}
```

Compilar el Proyecto:

dotnet build MiExportador.csproj (de esta forma podemos hacer un build sin hacerle referencia en la solución).

Desplegar el Exportador:

Copiar el archivo DLL generado en la carpeta Plugins del paquete WebApi, o la carpeta plugins dentro de la raíz del ejecutable.

El sistema lo detectará automáticamente en tiempo de ejecución mediante reflection.

Anexo

Especificación API

Descripción	Recurso	Verbo	URI	Header	Body	Response	Errores
Implementa una interfaz con sus métodos en una clase.	Implements	POST	api/v1/classes/{id}/implements	Authorization=false	{ "idInterface": "guid", "methods": [{ "name": "string", "privacy": "Public Private Protected ", "accessibility": "Normal Abstract Interface Sealed", "idReturnType": "guid", "parameters": [{ "name": "string", "idReference": "guid" }], "isStatic": "boolean", "isVirtual": "boolean", "isOverride": "boolean" }] }	(201 created). { "Message": "Class implemented successfully", "SimClass": { "SimClass": {} } }	400 Bad Request { "innerCode": 7, "message": "invalid JSON." } 404 Not Found : { "innerCode": 2, "message": "Class with id not found" } 409 Conflict : { "innerCode": 4, "message": "Class already implements this interface" } 500 internal server error { "innerCode": 5, "message": "Internal server error" }
Crea un nuevo namespace	Namespaces	POST	api/v1/namespaces	Authorization=false	{ "name": "string", "BaseNamespaceId": "Guid" }	(201 created). { "message": "Namespace created successfully", "namespaceResponse": { "id": "Guid", "name": "string", "baseNamespaceId": "Guid", "elements": [{ "SimClass": {} }] } }	400 Bad Request { "innerCode": 7, "message": "invalid JSON." } 404 Not Found { "innerCode": 2, "message": "Base namespace does not exist." } 500 internal server error { "innerCode": 5, "message": "Internal server error" }
Obtiene un namespace	Namespaces	GET	api/v1/namespaces/{id}	Authorization=false		(200 OK). { "id": "Guid", "name": "string", "baseNamespaceId": "Guid", "elements": [{ "SimClass": {} }] }	400 Bad Request . 500 internal server error { "innerCode": 5, "message": "Internal server error" }
Obtiene todos los namespaces	Namespaces	GET	api/v1/namespaces	Authorization=false		(200 OK.) [{ "id": "Guid", "name": "string", "baseNamespaceId": "Guid", "elements": [{ "SimClass": {} }] }]	500 internal server error { "innerCode": 5, "message": "Internal server error" }

						<pre> "ReferenceId": "Guid", "Instanceid": "Guid" }], "Invocations ": [{ "Id": "Guid", "IdReference": "Guid", "IdReturnType ": "Guid", "TypeReferenc e": "string", "MethodName ": "string", "Parameters" : [{ "Name ": "string", "ReferenceId ": "Guid", "Instanceid ": "Guid" }], "IsStatic ": "boolean", "IsVirtual ": "boolean", "IsOverride ": "boolean" }] } </pre>	
Obtiene un metodo	Methods	GET	api/v1/methods/{id}	Authorization=false		<pre> (200 OK.) { "Id": "Guid", "Name": "string", "IdClassOwn er": "Guid", "Privacy": "Public Private Protected ", "Accessibili ty": "Normal Abstract Interface Sealed", "ReturnTypel d": "Guid", "parameters": [{ "name": "string", "idReference": "Guid" }], "Variables": [[{ "Id": "Guid", "Name": "stri ng", "MethodId": " Guid", "ReferenceId ": "Guid", "Instanceid": "Guid" }], "Invocations ": [{ "Id": "Guid", "IdReference ": "Guid", "IdReturnTyp e": "Guid", "TypeReferen ce": "string", "MethodNam e": "string", "Parameters" : [{ "Name ": "string", "ReferenceId ": "Guid", "Instanceid ": "Guid" }] }, </pre>	<pre> 400 Bad Request{ "innerCode":7, "message":"invalid JSON."}, 404 Not Found{"innerCode": 2, "message": "SimClass does not exist."} 500 internal server error {"innerCode": 5,"message": "Internal server error"} </pre>

						"IsStatic" : "boolean", "IsVirtual" : "boolean", "IsOverride" : "boolean" }} }	
Actualiza una clase	Classes	PUT	api/v1/classes	Authorization=false	<pre>{ "Id": "Guid", "name": "string", "state": "Normal Interface Abstract Sealed", "IdBaseClass": "Guid", "implements": [{ "IdInterface": "Guid" }], "methods": [{ "Name": "string", "Privacy": "string (, 'Public', 'Protected', 'Private')", "Accessability": "string ('Interface', 'Normal', 'Abstract', 'Sealed')", "IdReturnType": "Guid", "Parameters": [{ "name": "string", "idReference": "Guid" }] }], "attributes": [{ "name": "string", "idReference": "Guid", "idInstance": "Guid" }] }</pre>	(200 OK.) { "Message": "Class updated successfully." "SimClass": { "Id": "Guid", "Name": "string", "IdBaseClass": "Guid", "State": "Normal Abstract Interface Sealed", "Attributes": [], "Implements": [], "Methods": [], "Namespaced": "Guid" } }	400 Bad Request { "innerCode": 7, "message": "invalid JSON."} 404 Not Found { "innerCode": 2, "message": "SimClass does not exist."} 409 Conflict: { "innerCode": 4, "message": "Class already has this name" } 500 internal server error { "innerCode": 5, "message": "Internal server error" }
Crea una clase	Classes	POST	api/v1/classes	Authorization=false	<pre>{ "Id": "Guid", "Name": "string", "State": "Normal Abstract Interface Sealed", "IdBaseClass": "string", "Methods": [{ "Name": "string", "Privacy": "Public Private Protected", "Accessability": "Normal Abstract Interface Sealed", "IdReturnType": "string", "Parameters": [{ "Name": "string", "IdReference": "string" }] }], "IsStatic": "boolean", "IsVirtual": "boolean", "IsOverride": "boolean" }, "Attributes": [{ "Name": "string", "IdReference": "string", "IdInstance": "string", "Privacy": "Public Private Protected", "IsStatic": "boolean" }], "IdBaseNamespace": "string" }</pre>	(201 created.) { "Message": "Class created successfully." "SimClass": { "Id": "Guid", "Name": "string", "IdBaseClass": "Guid", "State": "Normal Abstract Interface Sealed", "Attributes": [], "Implements": [], "Methods": [], "Namespaced": "Guid" } }	400 Bad Request { "innerCode": 7, "message": "invalid JSON."} 404 Not Found { "innerCode": 2, "message": "SimClass does not exist."} 409 Conflict: { "innerCode": 4, "message": "Class already has this name" } 500 internal server error { "innerCode": 5, "message": "Internal server error" }

Obtiene una clase	Classes	GET	Api/v1/classes/{id}	Authorization=false		(200 OK.) { "Id": "Guid", "Name": "string", "IdBaseClass": "Guid", "State": "Normal Abstract Interface Sealed", "Attributes": [], "Implements": [], "Methods": [], "NamespaceId": "Guid" }	400 Bad Request { "innerCode": 7, "message": "invalid JSON."}. 500 internal server error {"innerCode": 5,"message": "Internal server error"}
Obtiene todas las clases	Classes	GET	Api/v1/classes	Authorization=false		(200 OK.) [{ "Id": "Guid", "Name": "string", "IdBaseClass": "Guid", "State": "Normal Abstract Interface Sealed", "Attributes": [], "Implements": [], "Methods": [], "NamespaceId": "Guid" }]	500 internal server error {"innerCode": 5,"message": "Internal server error"}
Ejecuta un método dada una firma y referencia y instancia	Executions	POST	api/v1/executions	Authorization=false	{ "MethodName": "string", "Parameters": [{ "Name": "string", "IdReference": "Guid", "IdInstance": "Guid"}, "IdReturnType": "Guid", "IdReferenceType": "Guid", "IdInstanceType": "Guid" }	(200 OK.) { "Execution": "string" }	400 Bad Request { "innerCode": 7,"message": "invalid JSON."}. 404 Not Found {"innerCode": 2, "message": "SimClass does not exist."} 500 internal server error {"innerCode": 5,"message": "Internal server error"
Crea una invocacion	Invocations	POST	Api/v1/methods/{id}/invocations	Authorization=false	{ "IdReference": "Guid", "MethodName": "string", "IdReturnType": "Guid", "parameters": [{ "name": "string", "IdReference": "Guid" }, { "name": "string", "IdInstance": "Guid" }], "TypeReference": "Parameter LocalVariable StaticAttribute This Base StaticAttribute" }	(201 created.) { "message": "Invocation created successfully", "InvocationResponse": { "id": "Guid", "IdReference": "Guid", "IdReturnType": "Guid", "typeReference": "Parameter LocalVariable StaticAttribute This Base StaticAttribute", "methodName": "string", "parameters": [{ "name": "string", "IdReference": "Guid", "IdInstance": "Guid" }] } }	400 Bad Request { "innerCode": 7,"message": "invalid JSON."}. 404 not found { "innerCode": 2, "message": "Error creating invocation: Method does not exist." } 500 internal server error {"innerCode": 5,"message": "Internal server error"

						<pre> },], } } </pre>	
Crea un atributo	Attributes	POST	Api/v1/classes/{id}/attributes	Authorization=false	<pre> { "Name": "string", "IdReference": "Guid", "IdInstance": "Guid", "Privacy": "Public Private Protected", "IsStatic": boolean } </pre>	<pre> (201 created). { "Message": "Attribute created successfully", "Attribute": { "Id": "Guid", "Name": "string", "ReferenceId": "Guid", "InstanceId": "Guid", "Privacy": "Public Private Protected", "RelatedClassId": "Guid", "IsStatic": boolean } } </pre>	<pre> 400 Bad Request{ "innerCode":7,"message":"invalid JSON."}. 409 Conflict{ "innerCode":4, "message": "Attribute name 'counterValue' already in use in this class" } 500 internal server error {"innerCode":5,"message": "Internal server error"} </pre>
Actualiza un atributo	Attributes	PUT	api/v1/attributes	Authorization=false	<pre> { "Id": "Guid", "Name": "string", "IdReference": "Guid", "IdInstance": "Guid", "Privacy": "Public Private Protected", "IsStatic": boolean } </pre>	<pre> (200 OK). { "Message": "Attribute updated successfully", "Attribute": { "Id": "Guid", "Name": "string", "ReferenceId": "Guid", "InstanceId": "Guid", "Privacy": "Public Private Protected", "RelatedClassId": "Guid", "IsStatic": boolean } } </pre>	<pre> 400 Bad Request{ "innerCode":7,"message":"invalid JSON."}. 404 not found { "innerCode":2, "message": "Attribute not found," } 409 Conflict{ "innerCode":4, "message": "Attribute name 'counterValue' already in use in this class" } 500 internal server error {"innerCode":5,"message": "Internal server error"} </pre>
Obtiene un atributo	Attributes	GET	Api1/v1/attributes/{id}	Authorization=false		<pre> (200 OK) { "Id": "Guid", "Name": "string", "ReferenceId": "Guid", "InstanceId": "Guid", "Privacy": "Public Private Protected", "RelatedClassId": "Guid", "IsStatic": boolean } </pre>	<pre> 400 Bad Request{ "innerCode":7,"message":"invalid JSON."}. 500 internal server error {"innerCode":5,"message": "Internal server error"} </pre>

Informe de cobertura

Cobertura obtenida:

src	56.22%	
> Adapter	99.17%	
> BusinessLogic	94.97%	
> DataAccess	35.12%	
> Domain	97.05%	
> IAdapter\ Exceptions	100.00%	
> IBusinessLogic\ Exceptions	100.00%	
> Models	100.00%	
> WebApi	85.33%	

backend	83.41%	
src	56.22%	
> Adapter	99.17%	
> BusinessLogic	95.01%	
> DataAccess	35.12%	
> Domain	97.05%	
> IAdapter\ Exceptions	100.00%	
> Models	100.00%	
> WebApi	85.33%	

Desglose de los casos que no cumplen con el valor mínimo de cobertura:

DataAccess	35.12%	
ApikeyDataAccess.cs	100.00%	
> Context	100.00%	
ExecutionDataAccess.cs	94.24%	
> Migrations	0.00%	

La baja cobertura en este módulo se debe principalmente a que no se realizaron pruebas sobre la carpeta "Migrations", ya que contiene archivos autogenerados por Entity Framework Core, los cuales no se testean directamente debido a que no incluyen lógica de negocio. Esta carpeta posee una cobertura del 0%, lo que impacta negativamente en el porcentaje total de cobertura de la carpeta que la contiene.

WebApi	85.33%	
> Controllers	98.29%	
> Filters	100.00%	
Program.cs	0.00%	

Por otra parte, el archivo Program.cs también presenta una cobertura del 0%, lo cual disminuye el porcentaje general de la carpeta WebApi. Sin embargo, se considera que este archivo no tiene gran relevancia en términos de cobertura, ya que se limita a la configuración, y se ha decidido no incluirlo en las pruebas unitarias.

En conclusión, los valores bajos de cobertura observados no se deben a la falta de pruebas sobre la lógica desarrollada, sino a la inclusión de archivos que no afectan directamente el comportamiento funcional del sistema y que, por su naturaleza, no son candidatos directos para pruebas unitarias.

Assembly	Line Coverage	Branch Coverage	Health (over 90.0%)
Adapter	99.62%	95.00%	✓
BusinessLogic	96.61%	93.36%	✓
DataAccess	96.24%	87.20%	✗
Domain	97.62%	78.68%	✗
IAdapter	100.00%	100.00%	✓
Models	100.00%	50.00%	✗
WebApi	99.40%	100.00%	✓
-----	-----	-----	-----
Summary	98.50%	86.32%	✗

Lo que respecta a la cobertura generada por las github actions:

BranchCoverage:

Para la carpeta Models vemos que tiene un valor de **Branch Coverage** de 50%, esto se debe a que propiedades con validaciones parse de Guid.

Por otro lado, para los valores insuficientes de branch coverage de Domain y DataAccess se deben a que estos incluyen

funciones lambda con muchas ramas que no se podían testear todas las combinaciones.

Un ejemplo de esto es:

```
namespace Models.Request;

11 references
public class VariablesRequest()
{
    [Required(ErrorMessage = "Name is required.")]
    9 references
    public string? Name { get;
    [JsonIgnore]
    1 reference
    public Guid ReferenceId => Guid.TryParse(IdReference, out var guid) ? guid : Guid.Empty;

    [Required(ErrorMessage = "IdReference is required.")]
    9 references
    public string IdReference { get; set; } = string.Empty;

    [JsonIgnore]
    1 reference
    public Guid InstanceId => Guid.TryParse(IdInstance, out var guid) ? guid : Guid.Empty;

    [Required(ErrorMessage = "IdInstance is required.")]
    2 references
    public string IdInstance { get; set; } = string.Empty;
}
```

string VariablesRequest.IdReference { get; set; }

'IdReference' is not null here.

Branch #6 in Guid.TryParse(IdReference, out var guid)... was not covered.

Exclude From Coverage:

Es por lo mencionado anteriormente en la imagen que a alguna de las clases de Models les pusimos [ExcludeFromCoverage], para que una función que no se puede testear del sistema no nos impacte en la cobertura y nos permita mergear.

También incluimos en [ExcludeFromCoverage] el servicio de BussinesLogic de Reflection ya que manipulábamos carpetas y archivos del sistema ahí, y no podíamos testearlo con mocks ni MS Test.

La aplicación del atributo [ExcludeFromCodeCoverage] a los Helpers de Adapter se justifica principalmente porque esta clase contiene exclusivamente lógica de mapeo directa entre propiedades, sin incluir reglas de negocio complejas. La clase presenta una complejidad ciclomática prácticamente nula, sin contar con ramificaciones lógicas o decisiones condicionales complejas. Desarrollar pruebas unitarias para este tipo de mapeo resultaría redundante, ya que esencialmente duplicaría la implementación sin aportar valor significativo al proyecto.

Análisis de la cobertura:

Filter (e.g. text, !exclude, @tag)

609/609 13.8s

Tests (net8.0) 4.6s

- Tests.Adapter 478ms
- Tests.BusinessLogic 454ms
 - ApikeyServiceTest 6.0ms
 - ExecutionServiceTest 102ms
 - NamespaceServiceTest 15ms
 - SimAttributeServiceTest 41ms
 - SimClassServiceTest 105ms
 - SimMethodServiceTest 185ms
- Tests.DataAccess 3.4s
- Tests.Domain 35ms
 - AttributeTest 1.0ms
 - InvocationTest 0.0ms
 - LocalVariableTest 0.0ms
 - ParameterSignatureTest 0.0ms
 - ParameterTest 0.0ms
 - ReferenceAttributeTest 2.0ms

TEST COVERAGE

- backend 78.71%
- src 47.98%
- WebApi 85.33%
 - Controllers 98.29%
 - ClassAttributeController.cs 100.00%
 - ClassImplementController.cs 100.00%
 - ClassMethodController.cs 100.00%
 - InvocationsMethodController.cs 100.00%
 - MethodController.cs 100.00%
 - MethodExecutionController.cs 100.00%
 - NamespaceController.cs 84.62%**
 - ParameterMethodController.cs 100.00%

```

7 [ApiController]
8 [Route("api/v1/namespaces")]
9 public class NamespaceController(INamespaceAdapter namespaceAdapter) : ControllerBase
10 {
11     private readonly INamespaceAdapter _namespaceAdapter = namespaceAdapter;
12     [HttpGet]
13     public string Get()
14     {
15         // Branch #5 in return CreatedAtRoute("GetNamespaceById"... was not covered.
16         return CreatedAtRoute("GetNamespaceById", new { id = response.NamespaceId });
17     }
18
19     [HttpGet("{id}", Name = "GetNamespaceById")]
20     public IActionResult GetNamespaceById(Guid id)
21     {
22         return Ok(_namespaceAdapter.GetNamespaceById(id));
23     }
24
25     [HttpGet]
26     public IActionResult GetAllNamespaces()
27     {
28         // ...
29     }
30 }

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

DataAccess -> C:\Users\Betin\271395-294598-221509\backend\src\DataAccess\bin\Debug\net8.0\DataAccess.dll
 ServiceFactory -> C:\Users\Betin\271395-294598-221509\backend\src\ServiceFactory\bin\Debug\net8.0\ServiceFactory.dll
 WebApi -> C:\Users\Betin\271395-294598-221509\backend\src\WebApi\bin\Debug\net8.0\WebApi.dll
 Tests -> C:\Users\Betin\271395-294598-221509\backend\tests\bin\Debug\net8.0\Tests.dll

* Terminal will be reused by tasks, press any key to close it.

En general se logro el nivel esperado de cobertura mayor a 95% tenemos algunos casos que tienen menos de lo deseado por ejemplo en este caso que la cobertura llego a un 84% a pesar que se hizo con TDD no se testeo esa branch por lo que, bajo la cobertura, y al ser el único metodo baja bastante la misma.

Tests.BusinessLogic 454ms

- ApikeyServiceTest 6.0ms
- ExecutionServiceTest 102ms
- NamespaceServiceTest 15ms
- SimAttributeServiceTest 41ms
- SimClassServiceTest 105ms
- SimMethodServiceTest 185ms
- Tests.DataAccess 3.4s
- Tests.Domain 35ms
 - AttributeTest 1.0ms
 - InvocationTest 0.0ms
 - LocalVariableTest 0.0ms
 - ParameterSignatureTest 0.0ms
 - ParameterTest 0.0ms
 - ReferenceAttributeTest 2.0ms

TEST COVERAGE

- backend 78.71%
- src 47.98%
- BusinessLogic 95.49%
- DataAccess 27.84%
 - ApikeyDataAccess.cs 100.00%
 - ExecutionDataAccess.cs 94.24%
 - Migrations 0.00%
 - NamespaceDataAccess.cs 94.12%**
 - SimAttributeDataAccess.cs 100.00%
 - SimClassDataAccess.cs 95.88%

```

135 case ReferenceStatic rsv:
136     _context.Entry(rsv).Reference(r => r.Reference);
137     break;
138
139
140
141
142
143
144 return namespaces;
145
146 (field) readonly SimulatorDbContext NamespaceDataAccess._context
147 'context' is not null here.
148 public
149 {
150     // Branch #13 in return _context.SimNamespaces.Any(c => c... was not covered.
151     return _context.SimNamespaces.Any(c => c.Name == name);
152 }
153
154
155

```

PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL TEST RESULTS PORTS

DataAccess -> C:\Users\Betin\271395-294598-221509\backend\src\DataAccess\bin\Debug\net8.0\DataAccess.dll
 ServiceFactory -> C:\Users\Betin\271395-294598-221509\backend\src\ServiceFactory\bin\Debug\net8.0\ServiceFactory.dll
 WebApi -> C:\Users\Betin\271395-294598-221509\backend\src\WebApi\bin\Debug\net8.0\WebApi.dll
 Tests -> C:\Users\Betin\271395-294598-221509\backend\tests\bin\Debug\net8.0\Tests.dll

Otros ejemplos de cobertura baja es que no se logro cubrir ciertas branches de funciones de librerias de C# en NamespaceDataAccess.

Cobertura de líneas: Esto se cumplió ampliamente en el proyecto logrando una cobertura promedio total de 98,50%.

Configuración de la aplicación de angular:

La aplicación angular se encuentra configurada para recibir el endpoint de <http://localhost:5128>, dicho puerto está configurado en el ejecutable de la webApi del realese, también en dicha carpeta de publish se encuentra la carpeta plugins la que se necesita para cargar los transformadores de reflection.

Otro dato a saber es que como se mencionó en el foro, las ApiKeys están harcodeadas en el front end, las cuales son: 77777777-aaaa-1111-1111-111111111111 y 77777777-bbbb-1111-1111-111111111111